

Hands-On-Learning: Secure Authentication, Prompt Defense, and Monitoring for Healthcare APIs

Learning Objectives

By the end of this activity, you will be able to:

- Implement scope-based access control using OAuth2/JWT tokens.
- Apply a basic defense pipeline combining input sanitization, prompt isolation, and output filtering.
- Analyze sample healthcare access logs to detect authentication and prompt injection anomalies.
- Configure a simple security alert rule for suspicious authentication activity.



Description

In this hands-on practice, you will secure a healthcare API by implementing authentication checks, building prompt-injection defenses, and configuring a basic security alert.

Structure of the Activity

Title: Secure Authentication, Prompt Defense, and Monitoring for Healthcare APIs

Dataset Information:

File: m2_secure_healthcare_api_dataset_readme.md

Contents:

- Sample API access logs and authentication events
- Prompt-injection attempts
- Ten representative examples from a larger dataset of five thousand entries
- Key fields: endpoint, auth_method, status_code, prompt, blocked_reason, failure_reason

Tools Used:

→ **Python 3.10+**

→ **Local Jupyter Notebook** or any Python environment

→ Required libraries:

→ fastapi

→ python-jose[cryptography]

→ passlib[bcrypt]

→ prometheus-client

Setup Requirements:

Technical prerequisites

- Ability to run Python files or Jupyter notebooks
- Basic understanding of classes, functions, and dictionaries in Python

System requirements:

- **Required files in working directory** Minimum 4 GB RAM
- Stable environment capable of installing Python packages
- m2_secure_healthcare_api_dataset_readme.md for dataset reference

(Optional) Your personal Python file or notebook to write solutions

Scenario

You are the security engineer at HealthTech Inc., where an AI-driven healthcare API processes sensitive medical documents. Recent logs show failed authentications and attempted prompt injection attacks. Compliance officers require improved protection across authentication, input handling, and real-time monitoring. Your task is to strengthen the API's security posture while ensuring HIPAA-compliant behavior.

Objective

Implement and test a minimal security pipeline that validates authentication scopes, blocks prompt-injection attempts and triggers a basic alert for repeated authentication failures.



Getting Started:

- 1 Open your Python environment (Jupyter Notebook or .py file).
- 2 Place the dataset file `m2_secure_healthcare_api_dataset_readme.md` in your working directory for reference when extracting sample attack patterns.
- 3 Verify your environment by importing required libraries and printing a confirmation message such as "Environment ready".

Activity 1: Implement Scope-Based Authentication Validation

Objective

Write a function that validates OAuth2/JWT scopes and test it using sample tokens from the dataset context.



Input/Data:

- Required file: m2_secure_healthcare_api_dataset_readme.md
- No previous activity data required
- Essential imports only if needed:
Minimal imports if learner chooses to use simple dict-based tokens
import json

Steps to Complete

Step 1:

- Create two examples of token dictionaries that include a "scopes" field.
- Use one valid scope set and one invalid set (for example, missing "admin").

Step 2:

- Write a function `validate_scope(token, required_scope)` that checks whether a given scope exists inside the token's "scopes" list.

Step 3

→ Test the function by checking:

→ A token that does contain the required scope.

→ A token that does not contain the required scope.

Step 4

→ Print the results with simple pass/fail messages.

Expected Output

A function that returns True or False depending on whether the required scope exists.

A short printed summary indicating which tests passed and which failed.

Tokens represented as dictionaries or simple JSON-like structures.

Tips

If your token dictionary is missing a "scopes" key, return False to avoid crashes.

Keep scope names consistent ("patient:read", "patient:write", "admin").

Activity 2: Build a Basic Prompt-Injection Defense Pipeline

Objective

Objective: Create a three-step defense sequence that sanitizes input, isolates system instructions, and filters output for sensitive information.



Input/Data

Required file: m2_secure_healthcare_api_dataset_readme.md

Sample injection attempts extracted manually from the prompt field

Minimal imports (only if used):
import re

Steps to Complete

Step 1

Select a sample malicious prompt pattern from the dataset description (for example, attempts containing “ignore previous instructions” or “reveal patient data”).

Step 2

Write a simple sanitization function that removes or replaces known injection phrases.

Example approach: use `.replace()` or a small blocklist.

Step 3

Create a prompt isolation function that constructs two separate string blocks:

- A fixed system instruction (immutable)
- The user's sanitized input

Step 4

Write a simple output filter that scans the model output string and blocks patterns like emails, phone numbers, or IDs.

Example: detect numbers longer than six digits or patterns like "patient_id".

Step 5

Run the pipeline end-to-end using a sample injected query and a fake model output (e.g., "Patient ID: 1234567").

Print whether the output was blocked or allowed.

Expected Output

Sanitized user query with removed injection attempts

Structured prompt containing system role + user role

Filtered output that blocks sensitive strings

A short printed message indicating blocked or safe output

Tips

Use simple string operations; you do not need heavy NLP or ML.

If the output contains any sensitive pattern, block it immediately and return a safe message.

Test with at least one clearly malicious prompt and one benign prompt.

Activity 3: Configure a Minimal Security Alert for Authentication Failures

Objective

Define a lightweight alert rule that triggers when authentication failures exceed a threshold.



Input/Data

Required file: m2_secure_healthcare_api_dataset_readme.md

Data from Activity 1 (token validation result counts)

Minimal imports: import time

Steps to Complete

Step 1

Simulate repeated failed authentication attempts by running your `validate_scope()` function with missing/incorrect scopes several times.

Step 2

Create a counter that increments on each failed validation.

Step 3

Define an alert rule:

If failures in the last N attempts exceed a threshold (for example, 3 out of 5), print an alert message like:

"ALERT: Multiple authentication failures detected."

Step 4

Run the simulated sequence and verify whether the alert triggers.

Expected Output

A printed log of successes and failures

A final line indicating whether the alert condition was met

Clearly structured alert message when threshold is crossed

Tips

Use a simple loop to simulate multiple events.

When counting failures, ensure you reset or manage the counter carefully to avoid false alerts.

Keep the alert text short and actionable.

Compile Your Findings

Requirement 1: Authentication Scope Validation

Description: Show that you can implement and test scope-based access control using simple token structures.

Include:

Specific code/output required:

- A `validate_scope(token, required_scope)` function.
- Two sample tokens (one valid, one invalid).
- Printed pass/fail results for each test case.

Evidence of understanding:

- Short explanation (2–3 sentences) describing why scope-based validation strengthens healthcare API security.
- Mention how missing scopes should be treated as failures.

Requirement 2: Prompt-Injection Defense Pipeline

Description

Implement a working three-step defense pipeline that sanitizes user input, isolates system instructions, and filters sensitive output.

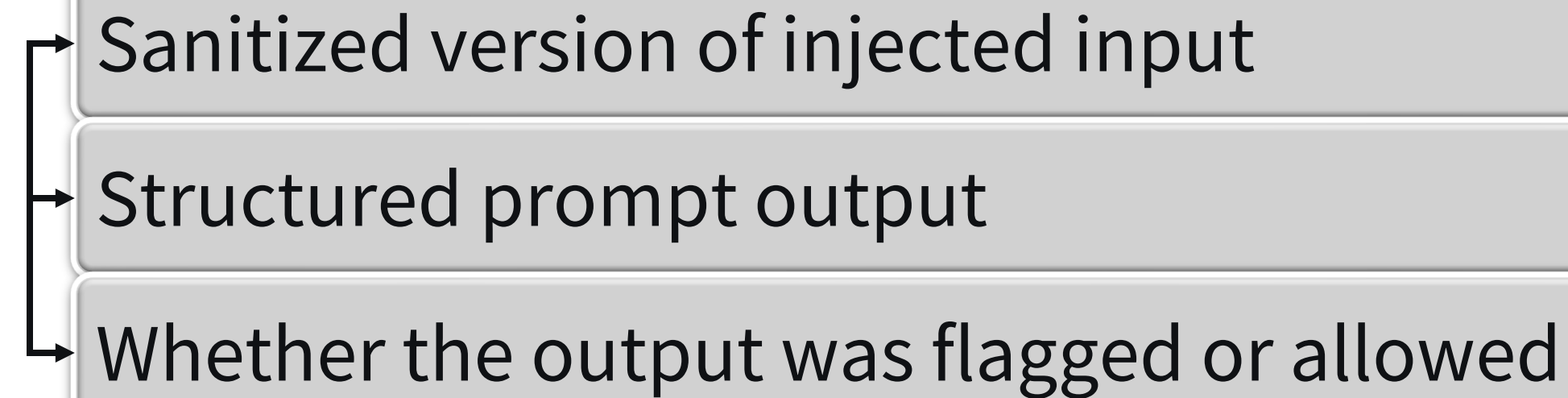
Include:

Implementation details:

- A `sanitize_input()` function removing injection phrases.
- An `isolate_prompts(system_instruction, user_input)` function returning a two-field structured prompt.
- A `filter_output(model_output)` function that blocks sensitive patterns.
- One injected prompt and one clean prompt used for testing.

Results/metrics:

- Printed demonstration showing:

- 
- Sanitized version of injected input
 - Structured prompt output
 - Whether the output was flagged or allowed

A brief statement (1–2 sentences) about detection accuracy based on your test examples.

Requirement 3: Authentication Failure Alert Rule

Description

Create a minimal alert mechanism that triggers authentication failures exceeding a defined threshold.

Include:

Analysis/interpretation:

A failure to counter simulation (e.g., 5 attempts) showing how many failures occurred.

Printed alert message when the threshold is exceeded.

A short explanation (2–3 sentences) on how such alerts help security teams detect suspicious activity early.

Business implications:

→ A concise interpretation of repeated authentication failures may signify (e.g., compromised credentials, brute-force attempts).

→ Why real-time alerts support HIPAA compliance and operational readiness.

Summary

This activity teaches **secure API development** through practical implementation of authentication validation, prompt-injection defenses, and basic security alerting.

You've learned how to **validate access scopes, block unsafe prompts, and detect suspicious authentication failures**—skills directly applicable in **real-world healthcare API security and HIPAA-compliant environments**.

Resources

Dataset: m2_secure_healthcare_api_dataset_readme.md

Documentation:

- OAuth2 Scopes (concept)
- Prompt Injection Prevention (general best practices)
- Basic Alerting Patterns (conceptual examples)

Share The Project

Document: Save the report in PDF format.

Upload: Share in the “Peer Review” area of the course shell with a brief description.

Instructions for Documenting and Sharing The Project for Peer Review

1. Document the Project

- Use word processing software to create your report.
- Ensure all sections of the project document structure are completed.
- Proofread and edit your report for clarity and accuracy.

2. Share the Project

- Save your report in PDF format.
- Upload your documents to the “Peer Review” area in the course shell.
- Provide a brief description of your project in the submission post.

Instructions for Documenting and Sharing The Project for Peer Review

3. Peer Review

- Review the projects submitted by your peers.
- Provide constructive feedback and comments on their work.